

1-Number of Zeros in a Given Array

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format First Line Contains Integer m – Size of array Next m lines Contains m numbers – Elements of an array

Output Format First Line Contains Integer – Number of zeroes present in the given array.

C Code

```
#include <stdio.h>

int countZero(int a[], int l, int r) {
    if (l > r)
        return 0;

    if (a[l] == 0)
        return r - l + 1;

    int mid = (l + r) / 2;

    if (a[mid] == 0)
        return countZero(a, l, mid - 1) + r - mid + 1;

    return countZero(a, mid + 1, r);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("%d", countZero(a, 0, n - 1));

    return 0;
}
```

Sample Input

```
5
1 1 1 0 0
```

Sample Output

```
2
```

2-Majority Element

Problem Statement

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1: Input: nums = [3,2,3] Output: 3

Example 2: Input: nums = [2,2,1,1,1,2,2] Output: 2

C Code

```
#include <stdio.h>

int majority(int a[], int l, int r) {
    if (l == r)
        return a[l];

    int mid = (l + r) / 2;
    int left = majority(a, l, mid);
    int right = majority(a, mid + 1, r);

    if (left == right)
        return left;

    int lc = 0, rc = 0;

    for (int i = l; i <= r; i++) {
        if (a[i] == left) lc++;
        if (a[i] == right) rc++;
    }

    return lc > rc ? left : right;
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("%d", majority(a, 0, n - 1));

    return 0;
}
```

Sample Input

```
7
2 2 1 1 1 2 2
```

Sample Output

```
2
```

3-Finding Floor Value

Problem Statement

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format First Line Contains Integer n – Size of array Next n lines Contains n numbers – Elements of an array Last Line Contains Integer x – Value for x

Output Format First Line Contains Integer – Floor value for x

C Code

```
#include <stdio.h>

int floorValue(int a[], int l, int r, int x) {
    if (l > r)
        return -1;

    int mid = (l + r) / 2;

    if (a[mid] == x)
        return a[mid];

    if (a[mid] < x) {
        int ans = floorValue(a, mid + 1, r, x);
        return ans == -1 ? a[mid] : ans;
    }

    return floorValue(a, l, mid - 1, x);
}

int main() {
    int n, x;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    printf("%d", floorValue(a, 0, n - 1, x));

    return 0;
}
```

Sample Input

```
6
1 3 5 7 9 11
8
```

Sample Output

```
7
```

4-Two Elements sum to x

Problem Statement

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format First Line Contains Integer n – Size of array Next n lines Contains n numbers – Elements of an array Last Line Contains Integer x – Sum Value

Output Format First Line Contains Integer – Element1 Second Line Contains Integer – Element2 (Element 1 and Element 2 together sums to value "x")

C Code

```
#include <stdio.h>

int findPair(int a[], int l, int r, int x, int *p, int *q) {
    if (l >= r)
        return 0;

    int sum = a[l] + a[r];

    if (sum == x) {
        *p = a[l];
        *q = a[r];
        return 1;
    }

    if (sum < x)
        return findPair(a, l + 1, r, x, p, q);

    return findPair(a, l, r - 1, x, p, q);
}

int main() {
    int n, x, p, q;

    scanf("%d", &n);
    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    if (findPair(a, 0, n - 1, x, &p, &q)) {
        printf("%d\n%d", p, q);
    } else {
        printf("No");
    }

    return 0;
}
```

Sample Input

```
5
1 2 4 7 9
11
```

Sample Output

```
2
9
```

5-Implementation of Quick Sort

Problem Statement

Write a Program to Implement the Quick Sort Algorithm

Input Format: The first line contains the no of elements in the list-n The next n lines contain the elements.

Output: Sorted list of elements

C Code

```
#include <stdio.h>

void quickSort(int a[], int low, int high) {
    if (low >= high)
        return;

    int i = low, j = high;
    int pivot = a[(low + high) / 2];

    while (i <= j) {
        while (a[i] < pivot)
            i++;

        while (a[j] > pivot)
            j--;

        if (i <= j) {
            int t = a[i];
            a[i] = a[j];
            a[j] = t;
            i++;
            j--;
        }
    }

    quickSort(a, low, j);
    quickSort(a, i, high);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

Sample Input

```
5
67 34 12 98 78
```

Sample Output

```
12 34 67 78 98
```